

CAP455: OBJECT ORIENTED PROGRAMMING USING C++

The Project Report For

Farm Produce Cold-Chain Tracker

Master of Computer Applications (MCA)
Academic Session: 2026–2027
Submission: CA1 Project Report

Submitted By:

HARSH THAKUR

Reg No.: 12613356

Roll No.: RD1P2635A03

Submitted To:

Mr. Adarsh Kumar Singh

GitHub Repo. Link.: https://github.com/Harsht1112/Farm-Produce-Cold--Chain-Tracker.git

TABLE OF CONTENTS

Content	Page Number
1. EXECUTIVE SUMMARY	**3**
2. PROBLEM STATEMENT & PROPOSED SOLUTION	**4**
3. ENTITY & MAIN CLASS DESIGN	**5**
4. UNIQUE KEY & DUPLICATE PREVENTION	**6**
5. DATA DICTIONARY: CLASS ATTRIBUTES	**7**
6. DATA DICTIONARY: FUNCTIONS	**8**
7. CONCEPT MAPPING	**10**
8. UML CLASS DIAGRAM	**12**
9. SYSTEM FLOWCHART	**13**
10. SOURCE CODE	**14**
11. OUTPUT	**20**
12. LIMITATIONS & FUTURE SCOPE	**23**

1. EXECUTIVE SUMMARY

The Farm Produce Cold-Chain Tracker is a console-based application developed in C++ to record and manage batches of agricultural produce stored in warehouses and cold-storage facilities.

The application uses Object-Oriented Programming concepts through a class named "Produce". Each object stores a batch ID, produce name, quantity in kilograms, storage temperature in degrees Celsius and storage location. All data members are private, and public member functions provide controlled access to the data.

The program supports adding, displaying, searching, updating and deleting produce batches. It validates user input to prevent invalid quantities, out-of-range temperatures, empty names, empty storage locations and duplicate batch IDs. A static data member maintains the total number of batches successfully added.

The project also demonstrates important C++ concepts, including encapsulation, arrays of objects, inline functions, static member functions, friend functions, object pointers, the "this" pointer, strings and reference parameters.

The program uses a fixed-size array that can store a maximum of five batches. All records are maintained in memory during execution and are not permanently saved after the program terminates.

2. PROBLEM STATEMENT & PROPOSED SOLUTION

2.1 Background and Problem Statement

Fresh produce such as tomatoes, potatoes and apples is perishable, and its quality depends strongly on how and where it is stored. Every batch arriving at a storage facility must be recorded together with its quantity, the temperature at which it is kept and the room or godown in which it is placed. When this is done manually, the accuracy of the register depends entirely on the person writing it.

Manual registers cause several difficulties. The same batch may be entered more than once, or two different batches may be given the same identification number, which makes it unclear which record is correct. Searching for one batch may require reading many pages. Entries can also contain impossible values, such as a negative quantity or a temperature of several hundred degrees, because nothing checks them at the time of writing. Physical registers can be misplaced or damaged, and separate copies may disagree with each other.

2.2 Proposed Solution

The proposed solution is a C++ console program that stores each batch as an object of the Produce class. The class groups the related details into one entity and provides member functions for accepting, validating, displaying and updating them. Private data members support encapsulation, and public functions form the controlled interface. A fixed-size array holds up to five Produce objects during one program run.

Before a batch is accepted, the program checks that the ID is positive and that it does not match any ID already stored. Each field is validated as it is entered: the name and location must not be empty, the quantity must be greater than zero, and the temperature must lie between -50 and 50 degrees Celsius. The program does not replace a full warehouse management system, but it demonstrates a structured first step using the C++ concepts required for CA1.

2.3 Project Objectives

- To represent produce batch information through a C++ entity class named Produce.
- To store, display, search, update and delete batch records held in an array of objects.
- To prevent duplicate batch IDs by checking existing objects before accepting a new record.
- To validate the name, quantity, temperature, location, batch count and IDs entered by the user.
- To demonstrate CA1 concepts including static members, inline and friend functions, arrays of objects, object pointers, the this pointer, strings and reference parameters.

3. ENTITY & MAIN CLASS DESIGN

3.1 Entity Overview

The application uses one main entity class named Produce. It represents one batch of produce together with the storage details recorded for it. Keeping all details in a single class keeps this first-phase program simple and allows array-of-objects and class concepts to be demonstrated without introducing inheritance or additional entities.

3.2 Produce Class Attributes

Attribute	Type	Description
batchID	int	Unique identification number of a batch; must be positive.
produceName	string	Name of the produce, for example Tomato or Potato.
quantity	float	Quantity of the batch in kilograms; must be greater than zero.
temperature	float	Storage temperature in degrees Celsius; allowed range -50 to 50.
storageLocation	string	Room, godown or cold store where the batch is kept.
totalBatches	static int	Class-level counter of batches successfully added.

3.3 Responsibilities of the Class

The Produce class accepts a new batch through addProduce(int id), which stores the ID supplied by main() and then reads and validates the remaining fields. The function getBatchID() returns the ID used by the duplicate check and by the search and delete operations, and getQuantity() returns the quantity. The function displayProduce() prints the full record, and updateProduce() re-enters all fields of an existing record with the same validation. The static function showTotalBatches() prints the class-level count, setBatchID() assigns a new ID, and the friend function showTemperature() prints the private temperature of a batch.

3.4 Encapsulation and Access Control

All five attributes and the static counter are declared private, so code outside the class cannot read or modify them directly. The main function works with a Produce object only through the public member functions, for example addProduce() to collect data and displayProduce() to show it. The validation logic is kept inside the class, so a record cannot be stored with an empty name, a non-positive quantity or an out-of-range temperature.

The friend function showTemperature() is a deliberate exception to private access. It is declared with the friend keyword inside the class and defined outside it. It is not a member function, but the class grants it permission to read the private member temperature. Its const reference parameter avoids copying the object and prevents modification through that parameter.

3.5 Scope of the Entity

One Produce object represents one batch. The program does not define a separate Storage or Supplier class, and it does not define a user-written constructor or destructor. This is a deliberate simplification for CA1; later phases may refine the model.

4. UNIQUE KEY & DUPLICATE PREVENTION

4.1 Unique Key

The data member batchID is the unique key that identifies each Produce record. It is an integer, and a valid ID must be greater than zero. The program reads an existing record's ID with getBatchID() during duplicate checking, searching and deletion.

4.2 Duplicate Checking Process

For the i-th batch, the program reads an ID inside a do-while loop. A while loop first ensures that the input is numeric and positive; otherwise it clears the error flag with cin.clear(), discards the rest of the line with cin.ignore(), and asks again. A boolean variable named duplicate is then set to false. A for loop runs from j = 0 up to j < i and compares the new ID with batches[j].getBatchID(). If the two are equal, duplicate is set to true, an error message is printed and the loop is stopped with break. The do-while loop repeats while duplicate is true, so the user is asked again until a unique ID is entered. Only then is addProduce(id) called for batches[i].

4.3 Pseudocode

```
START
Input n; repeat until 1 <= n <= 5
For i = 0 to n - 1:
    Repeat:
        Input id; repeat until id > 0
        duplicate = false
        For j = 0 to i - 1:
            If batches[j].getBatchID() == id:
                duplicate = true; display message; break
    Until duplicate == false
    batches[i].addProduce(id)
STOP
```

4.4 Why the Inner Loop Uses i

The inner loop condition is j < i because only the first i array positions contain batches that have already been registered. The remaining positions are unused capacity and hold no meaningful data. Checking only the occupied positions avoids comparing the new ID with uninitialized values.

4.5 Validation Boundary

Duplicate prevention covers only the batches entered during the current run. Because storage is in memory, all records and the duplicate-check history are lost when the program exits. Duplicate checking across separate runs would require file handling and is not claimed as a feature of this version.

5. DATA DICTIONARY: CLASS ATTRIBUTES

This data dictionary describes the variables declared in the Produce class. The sizes of int, float and char shown below are typical for a common g++ environment and may differ on other compilers or platforms. The size of a std::string object is implementation-dependent and is not the same as the length of the text it holds.

Variable	Data Type	Access	Typical Size	Purpose and Validation
batchID	int	private	4 bytes	Unique batch key; must be greater than zero.
produceName	string	private	Implementation-dependent	Name of the produce; must not be empty.
quantity	float	private	4 bytes	Quantity in kg; must be greater than zero.
temperature	float	private	4 bytes	Temperature in °C; must be from -50 to 50.
storageLocation	string	private	Implementation-dependent	Storage location; must not be empty.
totalBatches	static int	private	4 bytes	Shared counter incremented after each successful addProduce().

5.1 Local Variables Used in main()

Variable	Type	Purpose
batches	Produce[5]	Fixed-size array used for in-memory storage of batch objects.
n	int	Number of batches to be entered (1 to 5); reduced by one after a deletion.
i, j	int	Loop indices for entering, displaying, searching and shifting records.
id	int	Temporarily stores the Batch ID entered for a new batch.
duplicate	bool	Set to true when the entered ID already exists.
searchID	int	Batch ID entered by the user for searching.
found	bool	Indicates whether the searched batch was found.
ptr	Produce*	Points to the first element of the array (&batches[0]).
deleteID	int	Batch ID entered by the user for deletion.
foundDelete	bool	Indicates whether the batch to be deleted was found.
confirm	char	Stores the user's y/n answer for deletion confirmation.

5.2 Notes on Data Types

The program uses int for IDs and counts, float for quantity and temperature, bool for flags, char for the confirmation answer, and string for text. The static int totalBatches is shared by all Produce objects and is defined outside the class as int Produce::totalBatches = 0;.

6. DATA DICTIONARY: FUNCTIONS

6.1 Produce Class Functions

Function Name	Return	Concept	Purpose
addProduce(int id)	void	Member function; this pointer	Stores the ID, accepts and validates name, quantity, temperature and location, and increments totalBatches.
getBatchID()	int	Const member function (getter)	Returns the batch ID; used for duplicate check, search and delete.
getQuantity()	float	Inline function; const member	Returns the quantity of the batch.
displayProduce()	void	Const member function	Prints all details of the batch.
updateProduce()	void	Member function	Re-enters all fields of a batch with validation.
showTotalBatches()	void	Static member function	Prints the shared count of batches added.
setBatchID(int batchID)	void	this pointer	Assigns a new ID using this->batchID = batchID.
showTemperature(const Produce& p)	void	Friend function; reference parameter	Prints the private temperature of the given batch.

6.2 Static Data Definition

```
int Produce::totalBatches = 0;
```

This statement defines and initializes the static data member outside the class. The declaration inside the class describes the member, while this definition provides its storage and initial value.

6.3 Main Function Responsibilities

Operation	Main function behaviour
Initialization	Creates the array Produce batches[5] and reads n (1 to 5) with input validation.
Add batches	For each batch, reads a positive ID, checks duplicates, and calls addProduce(id).
Display all	Calls displayProduce() for every stored batch.
Search by ID	Reads an ID, loops through the batches, displays the match and calls showTemperature() and getQuantity().
Show total	Calls Produce::showTotalBatches() using the class name.
Pointer access	Creates Produce *ptr = &batches[0] and displays the first batch using ptr->displayProduce().
Delete	Reads an ID, asks for confirmation, shifts later records one position left and decreases n.

6.4 Function and Scope Clarification

The source does not define a user-written constructor or destructor, so the compiler-generated defaults are used. The inline function is getQuantity(), the static member function is showTotalBatches(), and the friend function is showTemperature(). The function updateProduce() is defined in the class as a complete update operation, but it is not called from main() in the current program.

7. CONCEPT MAPPING

7.1 Classes and Objects

A class is a user-defined type that groups data members and member functions, and an object is an instance of a class. The Produce class represents the information kept for one batch. In main(), the statement Produce batches[5]; creates an array of five Produce objects, each of which can hold one batch.

7.2 Encapsulation

Encapsulation combines data and the functions that operate on it within a class while controlling access to the data. In Produce, all attributes are private, and addProduce(), displayProduce(), updateProduce() and the getters form the public interface. The validation of name, quantity, temperature and location happens inside the class, so main() cannot place invalid values in an object.

7.3 Static Data Member and Static Function

The static data member totalBatches belongs to the class as a whole rather than to any single object. It is incremented at the end of each successful addProduce() call. The static function showTotalBatches() displays this shared value and is called with the class name as Produce::showTotalBatches(). The member is initialized outside the class with int Produce::totalBatches = 0;.

7.4 Inline Function

The function getQuantity() is declared with the inline keyword inside the class and returns the quantity of the current object. The inline keyword requests that the compiler substitute the function body at the call site, but the compiler makes the final decision. In main(), it is called when a searched batch is found.

7.5 Friend Function

The function showTemperature(const Produce &p) is declared as a friend inside the class and defined outside it. It is not a member function, yet it can read the private member temperature. It is called from main() after a successful search.

7.6 Array of Objects

The array batches contains five Produce objects and provides fixed-size in-memory storage. The variable n gives the number of positions currently in use. Registration fills positions 0 to n - 1, and the display, search and delete operations process only those positions. No dynamic allocation or STL container is used in this phase.

7.7 Pointers to Objects

The statement Produce *ptr = &batches[0]; declares a pointer to a Produce object and initializes it with the address of the first array element. The member function is then called through the

pointer with the arrow operator, as in ptr->displayProduce(). The pointer does not allocate memory; the array itself provides the storage.

7.8 this Pointer

The this pointer is available inside non-static member functions and points to the object on which the function is working. In addProduce(), this->batchID = id assigns the ID to the current object. In setBatchID(int batchID), the parameter has the same name as the data member, so this->batchID = batchID is needed to tell the member apart from the parameter. A static function cannot use this because it is not tied to a particular object.

7.9 std::string

The program includes the string header and uses the string type for produceName and storageLocation. Text is read with getline() so that entries containing spaces, such as "Cold Room A", are stored completely. Because the program uses using namespace std;, the type is written as string instead of std::string.

7.10 Reference Parameter and Call by Reference

The friend function accepts const Produce &p. The ampersand makes p refer to the original object instead of a copy, and const prevents the function from modifying it through that reference. By contrast, addProduce(int id) and setBatchID(int batchID) receive integers by value, so changes to the parameter do not affect the caller's variable.

7.11 Input Validation using Loops

Every numeric input is read inside a while loop with the condition !(cin >> value) combined with a range check. When the input is invalid, cin.clear() resets the failure state and cin.ignore(numeric_limits<streamsize>::max(), '\n') discards the unread characters, after which the user is asked again. Empty text is rejected with a while (text.empty()) loop. These loops make sure that only valid values are stored.

7.12 Deletion by Shifting

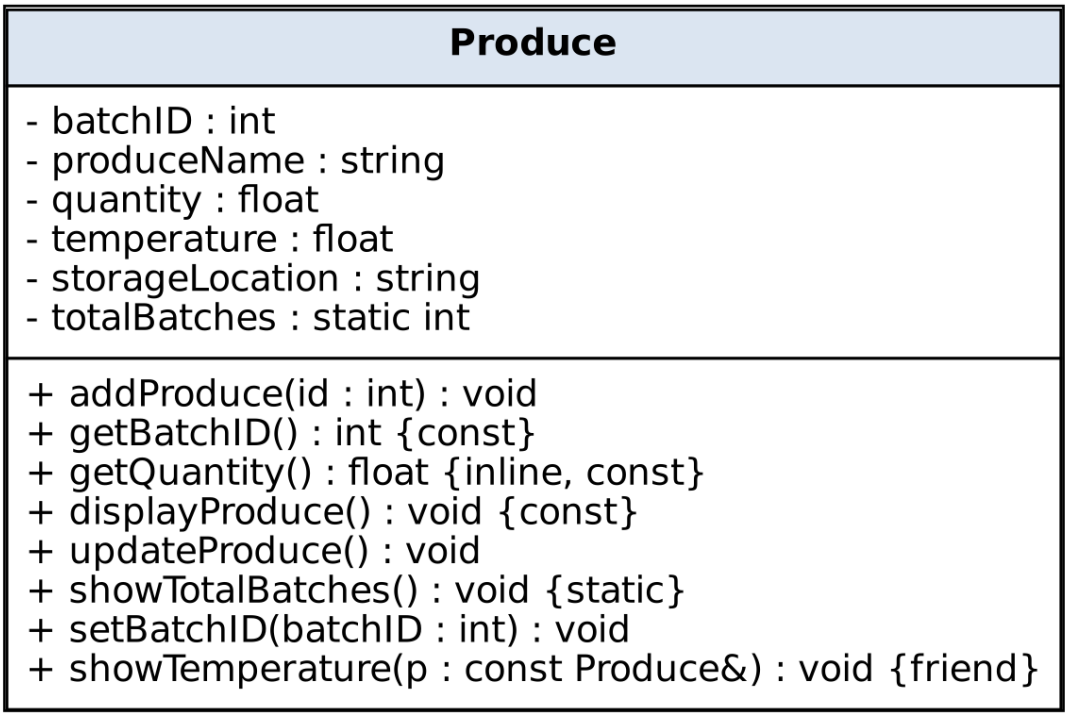
To delete a batch, the program finds its index i and, after confirmation, copies each later element one place to the left with batches[j] = batches[j + 1] for j from i to n - 2. The count n is then decreased by one. This uses the compiler-generated copy assignment operator of the Produce class.

7.13 Storage Limitation

The array stores data only in memory. Records remain available while the program runs, but they are not saved to a file, and closing the program removes them. File persistence is planned for a later phase and is not part of this version.

8. UML CLASS DIAGRAM

Design with one main entity class: Produce

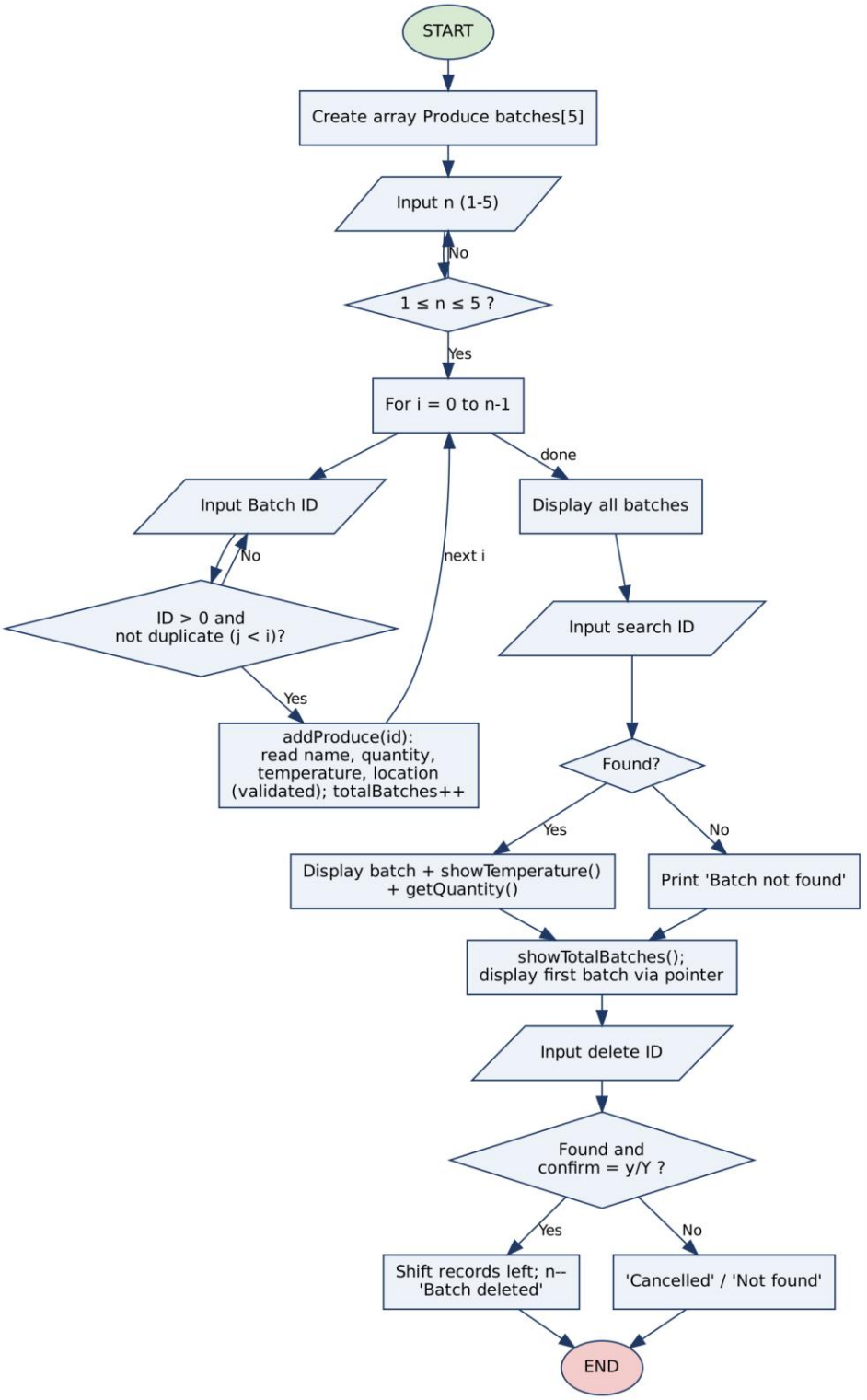


Relationship / implementation note

main() creates a fixed array Produce batches[5] and a Produce* pointer named ptr that points to its first element. Each array element represents one batch. The friend function showTemperature() is not a member of the class; it is listed in the diagram only to show its friend relationship. The program does not define any other entity class.

9. SYSTEM FLOWCHART

Batch entry, validation, search and delete operations



10. SOURCE CODE

```
#include <iostream>
#include <string>
#include <limits>
using namespace std;

class Produce
{
private:
    int batchID;
    string produceName;
    float quantity;
    float temperature;
    string storageLocation;

    // Static data member
    static int totalBatches;

public:

    // Function to add produce details
    void addProduce(int id)
    {
        this->batchID = id;

        cout << "Enter produce name: ";
        getline(cin, produceName);

        while (produceName.empty())
        {
            cout << "Name cannot be empty. Enter again: ";
            getline(cin, produceName);
        }

        // Validate quantity
        cout << "Enter quantity (kg): ";
        while (!(cin >> quantity) || quantity <= 0)
        {
            cout << "Invalid quantity! Enter a positive number: ";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }

        // Validate temperature
        cout << "Enter temperature (Celsius): ";
        while (!(cin >> temperature) ||
               temperature < -50 || temperature > 50)
        {
```

```
            cout << "Invalid temperature! Enter between -50 and 50: ";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Enter storage location: ";
        getline(cin, storageLocation);

        while (storageLocation.empty())
        {
            cout << "Location cannot be empty. Enter again: ";
            getline(cin, storageLocation);
        }

        // Increase total batch count
        totalBatches++;

        cout << "\nBatch added successfully!\n";
    }

    // Getter function
    int getBatchID() const
    {
        return batchID;
    }

    // Inline function
    inline float getQuantity() const
    {
        return quantity;
    }

    // Display batch details
    void displayProduce() const
    {
        cout << "\n------ Produce Details ------\n";
        cout << "Batch ID: " << batchID << endl;
        cout << "Produce Name: " << produceName << endl;
        cout << "Quantity: " << quantity << " kg" << endl;
        cout << "Temperature: " << temperature << " C" << endl;
        cout << "Storage Location: " << storageLocation << endl;
    }

void updateProduce()
{
    cout << "\n--- Update Produce Details ---\n";

    cout << "Enter new produce name: ";
```

```
    getline(cin, produceName);

    while (produceName.empty())
    {
        cout << "Name cannot be empty. Enter again: ";
        getline(cin, produceName);
    }

    cout << "Enter new quantity (kg): ";
    while (!(cin >> quantity) || quantity <= 0)
    {
        cout << "Invalid quantity! Enter again: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cout << "Enter new temperature: ";
    while (!(cin >> temperature) ||
           temperature < -50 || temperature > 50)
    {
        cout << "Invalid temperature! Enter again: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter new storage location: ";
    getline(cin, storageLocation);

    while (storageLocation.empty())
    {
        cout << "Location cannot be empty. Enter again: ";
        getline(cin, storageLocation);
    }

    cout << "\nBatch updated successfully!\n";
}


    // Static member function
    static void showTotalBatches()
    {
        cout << "\nTotal batches added: " << totalBatches << endl;
    }

    // Friend function declaration
    friend void showTemperature(const Produce &p);

    // this pointer example
```

```
    void setBatchID(int batchID)
    {
        this->batchID = batchID;
    }
};

// Definition of static data member
int Produce::totalBatches = 0;

// Friend function definition
void showTemperature(const Produce &p)
{
    cout << "Temperature using friend function: "
         << p.temperature << " C" << endl;
}

int main()
{
    Produce batches[5];

    int n;

    // Validate number of batches
    cout << "Enter number of batches (1-5): ";

    while (!(cin >> n) || n < 1 || n > 5)
    {
        cout << "Invalid number! Enter between 1 and 5: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }

    // Add batches
    for (int i = 0; i < n; i++)
    {
        int id;
        bool duplicate;

        cout << "\nEnter details for batch " << i + 1 << endl;

        do
        {
            cout << "Enter Batch ID: ";

            while (!(cin >> id) || id <= 0)
            {
                cout << "Invalid ID! Enter a positive number: ";
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
            }
```

```
        duplicate = false;

        // Check duplicate ID
        for (int j = 0; j < i; j++)
        {
            if (batches[j].getBatchID() == id)
            {
                duplicate = true;
                cout << "Batch ID already exists! Try another ID.\n";
                break;
            }
        }

    } while (duplicate);

    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    batches[i].addProduce(id);
}

// Display all batches
cout << "\n========== ALL BATCHES ==========\n";

for (int i = 0; i < n; i++)
{
    batches[i].displayProduce();
}

// Search batch by ID
int searchID;
bool found = false;

cout << "\nEnter Batch ID to search: ";

while (!(cin >> searchID) || searchID <= 0)
{
    cout << "Enter a valid positive Batch ID: ";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

for (int i = 0; i < n; i++)
{
    if (batches[i].getBatchID() == searchID)
    {
        cout << "\nBatch found!\n";
        batches[i].displayProduce();
        found = true;
```

```
            // Call friend function
            showTemperature(batches[i]);

            // Call inline function
            cout << "Quantity using inline function: "
                 << batches[i].getQuantity() << " kg\n";

            break;
        }
    }

    if (!found)
    {
        cout << "\nBatch not found!\n";
    }

    // Static function call
    Produce::showTotalBatches();

    // Pointer to object
    Produce *ptr = &batches[0];

    cout << "\nDisplaying first batch using pointer:\n";
    ptr->displayProduce();

    // UPDATE OPERATION
int updateID;
bool foundUpdate = false;

cout << "\nEnter Batch ID to update: ";
while (!(cin >> updateID) || updateID <= 0)
{
    cout << "Invalid ID! Enter a positive Batch ID: ";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

for (int i = 0; i < n; i++)
{
    if (batches[i].getBatchID() == updateID)
    {
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        batches[i].updateProduce();
        foundUpdate = true;
        break;
    }
}

if (!foundUpdate)
```

```
{
    cout << "Batch not found!\n";
}

    // DELETE OPERATION
    int deleteID;
    bool foundDelete = false;

    cout << "\nEnter Batch ID to delete: ";

while (!(cin >> deleteID) || deleteID <= 0)
{
    cout << "Invalid ID! Enter a positive Batch ID: ";
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

    for (int i = 0; i < n; i++)
    {
        if (batches[i].getBatchID() == deleteID)
        {
            char confirm;

            cout << "Are you sure you want to delete? (y/n): ";
            cin >> confirm;
            while (confirm != 'y' && confirm != 'Y' &&
       confirm != 'n' && confirm != 'N')
{
    cout << "Please enter y or n: ";
    cin >> confirm;
}

            if (confirm == 'y' || confirm == 'Y')
            {
                for (int j = i; j < n - 1; j++)
                {
                    batches[j] = batches[j + 1];
                }

                n--;

                cout << "Batch deleted successfully!\n";
            }
            else
            {
                cout << "Deletion cancelled.\n";
            }

            foundDelete = true;
            break;
```

```
        }
    }

    if (!foundDelete)
    {
        cout << "Batch not found!\n";
    }
    // Display batches after deletion
cout << "\n========== REMAINING BATCHES ==========\n";

for (int i = 0; i < n; i++)
{
    batches[i].displayProduce();
}
if (n == 0)
{
    cout << "No batches available.\n";
}

    return 0;
}
```

11. OUTPUT

11.1 Test Case 1: Normal Run with Duplicate ID

Three batches are entered. The ID 101 is entered again for batch 3 and is rejected. Batch 102 is then searched and deleted. (User input is shown after each prompt.)

```
Enter number of batches (1-5): 3

Enter details for batch 1
Enter Batch ID: 101
Enter produce name: Tomato
Enter quantity (kg): 50.5
Enter temperature (Celsius): 8
Enter storage location: Cold Room A

Batch added successfully!

Enter details for batch 2
Enter Batch ID: 102
Enter produce name: Potato
Enter quantity (kg): 120
Enter temperature (Celsius): 15
Enter storage location: Godown B

Batch added successfully!

Enter details for batch 3
Enter Batch ID: 101
Batch ID already exists! Try another ID.
Enter Batch ID: 103
Enter produce name: Apple
Enter quantity (kg): 75.25
Enter temperature (Celsius): 4
Enter storage location: Cold Room C

Batch added successfully!

========== ALL BATCHES ==========

------ Produce Details ------
Batch ID: 101
Produce Name: Tomato
Quantity: 50.5 kg
Temperature: 8 C
Storage Location: Cold Room A

------ Produce Details ------
Batch ID: 102
Produce Name: Potato
Quantity: 120 kg
Temperature: 15 C
Storage Location: Godown B

------ Produce Details ------
Batch ID: 103
Produce Name: Apple
Quantity: 75.25 kg
Temperature: 4 C
Storage Location: Cold Room C

Enter Batch ID to search: 102

Batch found!

------ Produce Details ------
```

```
Batch ID: 102
Produce Name: Potato
Quantity: 120 kg
Temperature: 15 C
Storage Location: Godown B
Temperature using friend function: 15 C
Quantity using inline function: 120 kg

Total batches added: 3

Displaying first batch using pointer:

------ Produce Details ------
Batch ID: 101
Produce Name: Tomato
Quantity: 50.5 kg
Temperature: 8 C
Storage Location: Cold Room A

Enter Batch ID to delete: 102
Are you sure you want to delete? (y/n): y
Batch deleted successfully!
```

11.2 Test Case 2: Input Validation

An invalid batch count (9), a negative quantity (-5), a non-numeric quantity (abc) and an out-of-range temperature (80) are entered and rejected. A search for a missing ID (999) and a deletion attempt for the same ID report "Batch not found!".

```
Enter number of batches (1-5): 9
Invalid number! Enter between 1 and 5: 2

Enter details for batch 1
Enter Batch ID: 201
Enter produce name: Banana
Enter quantity (kg): -5
Invalid quantity! Enter a positive number: abc
Invalid quantity! Enter a positive number: 30
Enter temperature (Celsius): 80
Invalid temperature! Enter between -50 and 50: 12
Enter storage location: Cold Room D

Batch added successfully!

Enter details for batch 2
Enter Batch ID: 202
Enter produce name: Onion
Enter quantity (kg): 40
Enter temperature (Celsius): 10
Enter storage location: Godown E

Batch added successfully!

========== ALL BATCHES ==========

------ Produce Details ------
Batch ID: 201
Produce Name: Banana
Quantity: 30 kg
Temperature: 12 C
Storage Location: Cold Room D

------ Produce Details ------
Batch ID: 202
Produce Name: Onion
Quantity: 40 kg
Temperature: 10 C
```

```
Storage Location: Godown E

Enter Batch ID to search: 999

Batch not found!

Total batches added: 2

Displaying first batch using pointer:

------ Produce Details ------
Batch ID: 201
Produce Name: Banana
Quantity: 30 kg
Temperature: 12 C
Storage Location: Cold Room D

Enter Batch ID to delete: 999
Batch not found!
```

12. LIMITATIONS & FUTURE SCOPE

12.1 Current Limitations

- The program is not menu-driven; it runs through a fixed sequence of add, display, search, pointer display and delete.
- The function updateProduce() is implemented but is never called from main(), so batches cannot be updated during a run.
- The static counter totalBatches is not decreased when a batch is deleted, so it shows the number of batches added, not the number currently stored.
- The delete ID and the y/n confirmation are read without validation.
- The array holds at most five batches, and records are lost when the program ends.

12.2 Future Scope

Later phases can add a menu loop so that registration, update and deletion can be repeated, save records to a file, use constructors and a destructor, overload operators for comparing batches, derive specialised classes such as perishable and non-perishable produce, add exception handling, and replace the fixed array with an STL vector.

12.3 Conclusion

The Farm Produce Cold-Chain Trackerprogram meets the CA1 objectives. It models a batch as a class with private data, validates every input, prevents duplicate IDs, and demonstrates static, inline and friend functions, arrays of objects, object pointers, the this pointer, strings and reference parameters in one working program.